

Strings in Python

part one

1. What are strings (and string documentation)

String documentation

Strings are one of Python's *Built-in Types*

You find detailed the documentation here among other built-in types

(e.g. other built-in types are Numeric types (int, float) and booleans):

https:

`//python.readthedocs.io/en/latest/library/index.html`

Then choose: Text Sequence Type - str

To reach: `https://python.readthedocs.io/en/latest/library/stdtypes.html#text-sequence-type-str`

- Built-in Types
 - Truth Value Testing
 - Boolean Operations — and, or, not
 - Comparisons
 - Numeric Types — int, float, complex
 - Boolean Type - bool
 - Iterator Types
 - Sequence Types — list, tuple, range
 - Text and Binary Sequence Type Methods Summary
 - Text Sequence Type — str
 - Binary Sequence Types — bytes, bytearray, memoryview
 - Set Types — set, frozenset
 - Mapping Types — dict

Python standard library string documentation

4.7. Text Sequence Type — `str`

Textual data in Python is handled with `str` objects, or *strings*. Strings are immutable [sequences](#) of Unicode code points. String literals are written in a variety of ways:

- Single quotes: `'allows embedded "double" quotes'`
- Double quotes: `"allows embedded 'single' quotes"`.
- Triple quoted: `'''Three single quotes'''`, `"""Three double quotes"""`

Triple quoted strings may span multiple lines – all associated whitespace will be included in the string literal.

Immutable means that a string cannot be changed (in contrast to e.g. a list, which is the topic for another lecture.).

Since a string is a type of immutable *sequence*, many of the standard sequence operations can be used on strings, e.g. `'+'`, but not those that change the string. (More on that later.)

2. Hello world again!

Different string types

Very standard

```
1 print("Hello world! It's nice today")
```

With single quotes (and then you need backspace in front of')

```
1 print('Hello world! It\'s nice today')
```

Triple quotes (to allow for multiple lines)

```
1 print("""  
2 Hello world!  
3 It's nice today  
4 """)
```

Newline in string

```
1 print("Hello world!\nIt's nice today")
```

Checking type for variables

The function `type()` can be used for checking the type.

```
1 my_str = "Hello world!"  
2 print(type(my_str))
```

`<class 'str'>`

Concatenating strings

With '+' two strings can be concatenated.

```
1 my_str = "Hello world!"
2 my_str_2 = "It's nice today!"
3 print(my_str + " " + my_str_2)
```

Hello world! It's nice today! But you can't concatenate a string and an another type, e.g. int. Then you get a `TypeError: can only concatenate str (not "int") to str`

You need to convert the int to a string first:

```
1 my_str = "Hello world!"
2 my_int = 1
3 print(my_str + " " + str(my_int))
```

Hello world! 1

3. Indexing and slicing strings

Indexing strings

As other *sequences* (e.g., lists, which we will talk about later), strings can be indexed. because strings are type of sequence

Important to remember:

You always start counting at 0

```
1 alphabet_str = "abcdefghijklm"  
2 print(alphabet_str[0])  
3 print(alphabet_str[1])
```

a

b

Indexing strings

One more example

Important to remember:

You always start counting at 0

```
1 greeting = "Hello?"  
2 print(greeting[5])
```

?

Indexing strings

If you provide an index that is not in the string you will get an

IndexError. if you go out of index, like if it's outside of string

Important to remember:

You always start counting at 0

```
1 alphabet_str = "abcdefghijklm"
2 print(alphabet_str[0])
3 print(alphabet_str[1])
4 print(alphabet_str[12])
5 print(alphabet_str[13])
```

a

b

m

IndexError: string index out of range

Slicing strings

Strings can also be sliced.

To get 'bcd', you start with the index for 'b', i.e. 1, and end with the index **after** 'd', i.e. 4

(remember that indexing starts at 0)

```
1 alphabet_str = "abcdefghijklm"  
2 print(alphabet_str[1:4])
```

bcd

Slicing strings

from beginning or towards the end

You can also start from the beginning or continue to the end

```
1 alphabet_str = "abcdefghijklm"  
2 # From the start  
3 print(alphabet_str[:3])  
4  
5 # Towards the end  
6 print(alphabet_str[5:])
```

abc

fghijklm

Indexing strings with negative indexes

You can also use negative indexes. Then you will start counting backwards.

```
1 alphabet_str = "abcdefghijklm"  
2 print(alphabet_str[-1])
```

m

String are immutable

There is an important difference between a list and a string:

Strings are immutable, and therefore cannot be changed.

```
my_alphabet_string = "abcdefghijklm"
```

```
my_alphabet_string[3] = "a"
```

The code will raise a **TypeError**:

```
TypeError: 'str' object does not support item  
assignment
```

String are immutable

```
alphabet = "abcdefghijklm"
```

```
alphabet[3] = "D"
```

The code will raise a `TypeError`:

```
TypeError: 'str' object does not support item  
assignment
```

If you want to change the content of `alphabet`, you need to assign it to a new value.

```
1 alphabet = "abcdefghijklm"  
2 alphabet = alphabet[:3] + "D" + alphabet[4:]  
3 print(alphabet)
```

```
abcDefghijklm
```

4. Looping over strings

Looping over strings

Looping over strings

With the keywords *for* and *in* you can loop over a string

```
1 for c in "Hello!":  
2     print(c)
```

H
e
l
l
o
!

Looping over strings

Another example:

```
1 alphabet_str = "abcdefghijklm"  
2 for ch in alphabet_str:  
3     print(ch)
```

a
b
c
d
e
f
g
h
i
j
k
l
m

5. Checking if something is in a string

Checking if something is in a string

Also the other usage of the keyword `in` works with strings:

```
1 def check_for_exclamation_mark(input_str):  
2     if '!' in input_str:  
3         print(input_str, "contains exclamation mark")  
4     else:  
5         print(input_str, "does not contain exclamation mark")  
6  
7 check_for_exclamation_mark("Hello word!")  
8 check_for_exclamation_mark("Hello word")
```

Hello word! contains exclamation mark

Hello word does not contain exclamation mark

Checking if something is in a string

Another example

```
1 greeting = "Hello world!"
2
3 if "world" in greeting:
4     print("The word 'world' was found")
5 else:
6     print("The word 'world' was NOT found")
```

The word 'world' was found

6. Built-in functions applied on strings

Reflections

on the built-in functions shown so far

`print()`, `type()`, `str()` are all functions of the same type. They are examples of *built-in* functions.

Python doesn't have very many built-in functions. You find a list of them here:

<https://docs.python.org/3/library/functions.html>.

Some of them can be applied on strings.

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available listed here in alphabetical order.

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

Reflections

on built-in functions

- ▶ `print()` takes objects to print as parameter (among other parameters), prints them and returns `None`.
- ▶ `type()` takes an object as parameter and returns its type (an instance of the `Type` class). Can be good for debugging.
- ▶ `str()` takes an object as parameter and returns a string.

More built-in functions that can be applied on strings

len()

len() can be used to find the length of a string

```
1 print(len("Hello world!"))
```

12

This is what the Python documentation says about len()

len(s)

Return the length (the number of items) of an object. The argument may be a sequence (such as a string, bytes, tuple, list, or range) or a collection (such as a dictionary, set, or frozen set).

More built-in functions that can be applied on strings

`len()`

We can make use of `len()` to avoid using an index that does not exist.

```
1 def index_safe(s, index):
2     if index < len(s):
3         return s[index]
4     else:
5         return None
6
7 alphabet_str = "abcdefghijklm"
8 print(index_safe(alphabet_str, 12))
9 print(index_safe(alphabet_str, 13))
```

m

None

Reflections

on built-in functions used on strings

Note that all examples so far have been on built-in *functions*.

- ▶ The built-in functions showcased can be applied on many different types of objects.
- ▶ They all take the strings as arguments, e.g., `len(my_str)` or `len('Hello world')`.

Now we will go on to look at **string *methods***. They are specific to **string objects and are called using *dot notation***.